

Hardware Design Document

Sentinel Final Version

Bryan Zhang, Bruce Zhang

TEJ4M, Mr. Roller

Friday, June 19, 2026

Table of Contents

Introduction.....3

PROJECT

OVERVIEW.....

Development Process and Consulted Resources.....

Block

Diagram.....4

User’s Guide.....5

APPENDIX A: Beta Testing and Recording—Test Results7

Testing of Functional Requirements.....

Testing of Hardware Requirements.....

Testing of User Interface Requirements.....

Testing of Hardware Interface Requirements.....

APPENDIX B: Reflection of

Learning.....18 APPENDIX C:

Code.....18

Tank

Code.....

Controller Code.....

Introduction

Project Overview

The Sentinel is a remotely operated robotic toy tank designed to combine entertainment and practical engineering in a single platform. Intended for hobbyists, students, and anyone curious about robotics, the Sentinel replicates many features of a real tank: rubber-tracked drive powered by two independent 6V JGB37-550 DC gear motors, a continuously rotating turret driven by a 28BYJ-48 stepper motor, an MG996R servo-controlled barrel with adjustable elevation, and a flywheel-based dart launcher capable of firing Nerf darts from a standard magazine. The tank is constructed primarily from 3D-printed PLA components, housing all electronics within a compact chassis powered by a 7.4V 2S Li-ion battery.

The operator controls the tank through a wired Xbox One controller attached to a custom 3D-printed handheld controller unit, which communicates wirelessly with the tank over a 2.4 GHz nRF24L01+PA+LNA radio link. Simultaneously, the operator receives a live first-person video feed transmitted wirelessly at 5.8 GHz from a RunCam Phoenix 2 FPV camera mounted on the tank to a SoloGood FPV monitor attached to the controller—allowing the tank to be driven effectively from far away and out of direct line of sight. The controller unit is built around an Arduino Uno microcontroller paired with a USB Host Shield, while the tank itself is controlled by a second Arduino Uno responsible for all motor, servo, and actuator outputs.

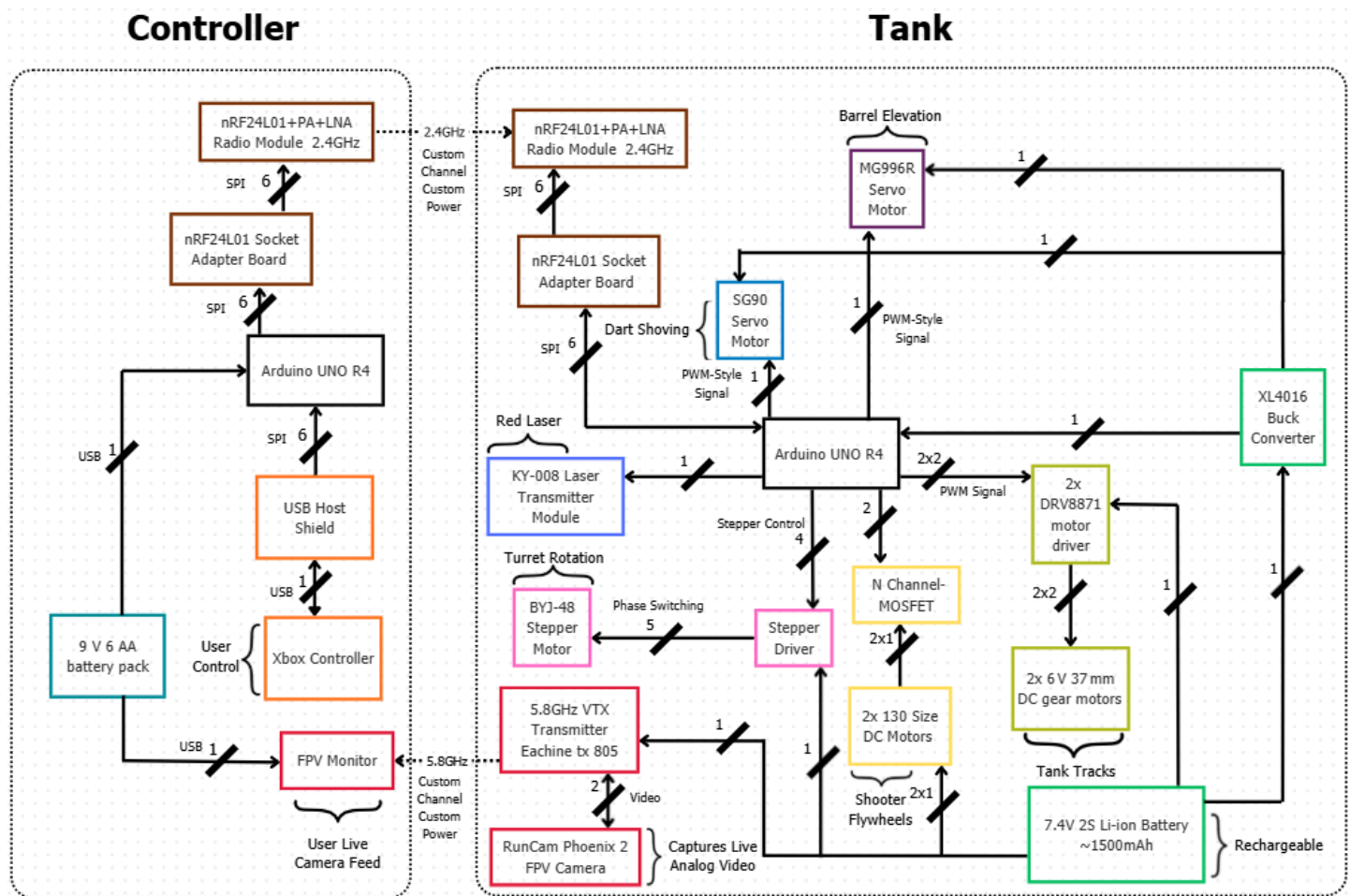
A full Hardware Requirements Specification (HRS) was written prior to this document and is available for reference, detailing all functional, hardware, and interface requirements agreed upon with the project stakeholder. The Preliminary Design Document (PDD) builds on those requirements with detailed sub-circuit descriptions, interface definitions, component-level calculations, and a project schedule covering assembly through to final testing. This Hardware Design Document (HDD) reflects the build process and changes we made to our design.

Development Process and Consulted Resources

Developed by Bruce Zhang and Bryan Zhang, these two friends were responsible for the design, assembly, wiring, and coding of the project. This process was lengthy, tedious, and involved a lot of help from external sources. For the design, the inspiration for this project came from toys and projects posted on Reddit and YouTube, which had much more complex and cool designs but demonstrated the potential and possibilities for the project. Alvin Ren and Felix Shen, two close friends, printed the parts for the assembly, using their 3D printers at home. Other mentors and friends were consulted for guidance, like Mr. Roller, who also provided parts in addition to his expertise. For parts of the design, assembly, and programming, AI agents like ChatGPT, Gemini, and Claude were used to aid and fast-track the development process. For example, Gemini was often used to generate programs to test individual components, enabling faster, more rigorous testing. Likewise, ChatGPT was often consulted for advice and guidance: making the design from scratch and finding the perfect components was often risky, so it helped with research and confirming design choices. Additionally, some firmware and third-party programs were required for the project. The CAD of the project was created in Onshape, and the code runs on the Arduino IDE, using common third-party libraries like the AccelStepper, RF24, and

XBOXONE (from USB Host Shield 2.0) libraries. Some of these libraries provided example sketches, too, which enabled quick, efficient debugging, testing, and even learning.

Block Diagram



User's Guide

Toggle Switch

This switch is located on the tank chassis and turns the entire tank system on or off.

Rechargeable Battery

This plug is located next to the toggle switch on the tank chassis and allows users to recharge the Sentinel's 7.4 lithium-ion battery with an XH-3Pin charger.



Plug-In Battery

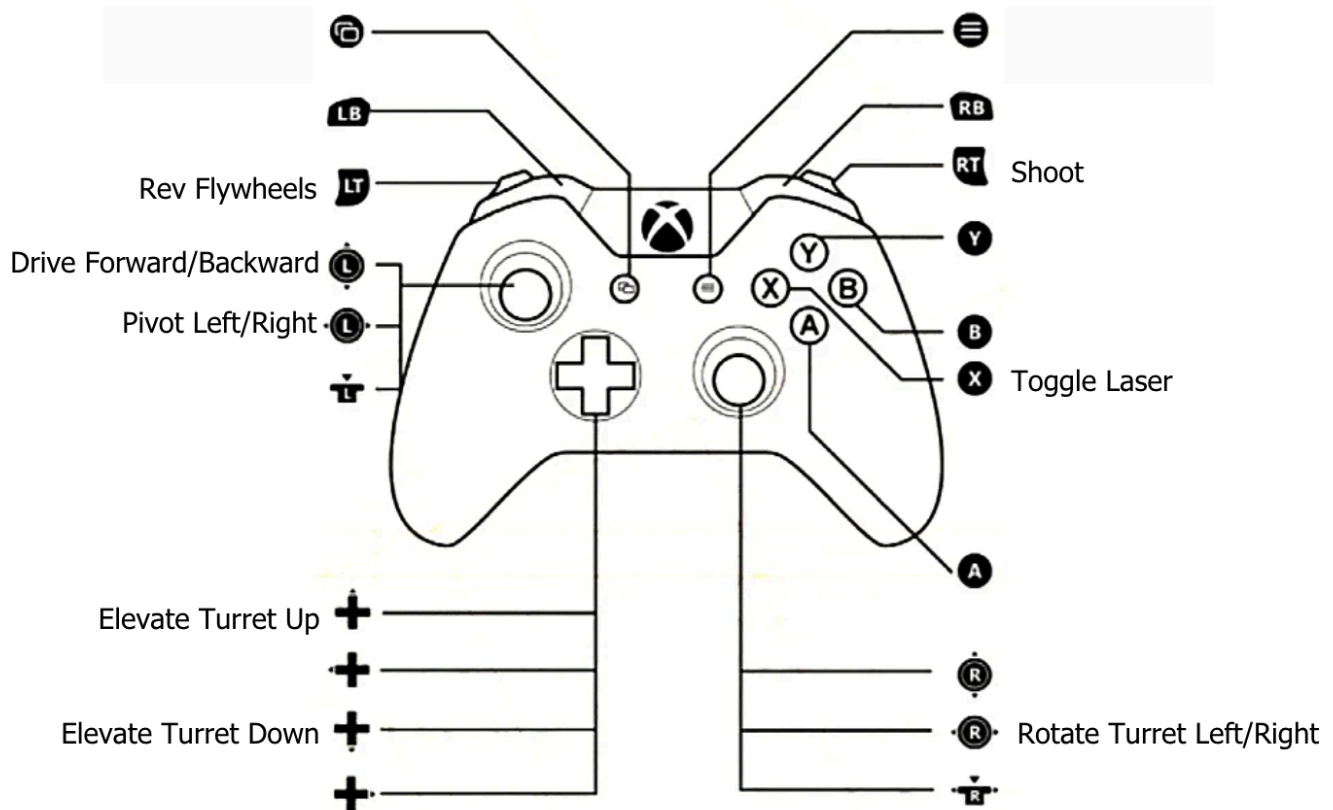
This power supply activates the Xbox controller and the radio system. Plug in the barrel jack into the Arduino Uno R3 to initiate radio communication. The 6 AA batteries are non-rechargeable and may need to be replaced.

Monitor Power

The Power button is used to turn on and off the monitor. The monitor has its own rechargeable, internal power supply. Charge the monitor through the USB micro port on the side of the monitor.

Monitor Connection

The Search button is used to automatically search for the tank's video stream. Once the tank is powered, the Search button allows the monitor to connect to the tank's radio channel and stream the live video. The CH+ or Band+ buttons may be used for further customization of the connection.



Left Joystick (Driving)

The left joystick controls the tank's movement using an arcade drive system. Moving the joystick forward and backward controls the speed and direction of both tracks, while moving it left or right controls turning. Combining both axes allows the tank to drive and turn simultaneously. A deadband and exponential response curve are used to improve control and prevent joystick drift.

Right Joystick (Turret Rotation)

The right joystick controls the turret's rotation. Moving the joystick left or right rotates the turret. Deadband and exponential controls improve precision.

Left Trigger (Flywheel Control)

The left trigger controls the flywheel motors used to launch darts. The farther the trigger is pressed, the faster the flywheels spin using PWM speed control. A deadband and response curve are included to provide smoother operation.

Right Trigger (Firing System)

The right trigger activates the dart-pushing servo to fire darts. The system only allows firing when the flywheels are already spinning, preventing jams and ensuring proper operation.

X Button (Laser Toggle)

The X button turns the turret-mounted laser on or off. The laser can be used to indicate where the turret is aiming.

Directional Pad (Barrel Elevation)

The D-Pad controls the barrel elevation. A quick press up or down moves the barrel up or down by increments of 3° . Holding a D-Pad direction continuously elevates the barrel at a constant speed.

Appendix A: Beta Testing and Recording – Test Results

Testing of Functional Requirements

Requirement Specification Reference	Requirement	Comments	Hardware Components Involved in Testing	Status	Date Tested
HRS-3.1.1	The controller shall permit the user to remotely control the tank's movements and features within an indoor range of at least 50 m	Control should be reliable: low latency, correct, long range, and without frequent disconnection/distortions	Xbox One controller (U5) Arduino Uno R3 microcontroller (U2 & U11) nRF24L01+ PA/LNA radio module (U6 & U15) USB Host Shield (U3)	Pass with Modification Control is reliable, low latency, correct but range is extremely limited	06/18
HRS-3.1.2	The controller shall provide a live video feed from the tank to a user within an indoor range of at least 50 m	Video should be reliable: low latency, correct, long range, and without frequent disconnection/distortions	RunCam Phoenix 2 (U12) Eachine TX805 5.8 GHz VTX (U9) Sologood FPV monitor (U1)	Pass with modification The video is low latency and correct. The range requirement was not met as the video could only reach a range of around 15m	06/18
HRS-3.1.3	The tank shall be able to move at different speeds and directions: it shall be able to go forward and backward, along with turning left and right by pivoting with tank drive mechanics.	Tank drive mechanics: turning and driving by controlling either left or right sides of the tank treads as separate units. E.g., turn left on the spot by moving left tracks backwards and right tracks forwards at full speeds	Arduino Uno R3 microcontroller (U11) JGB37-550 37mm motors (M4 & M6) DRV8871DDA DC Motor Driver (U14 & U17)	Pass Controlled by an arcade drive system, controlling the tank drive on one joystick	06/18
HRS-3.1.4	The tank shall be able to fire a Nerf foam dart at a distance of at least		Size-130 DC motors (M5 & M7) Arduino Uno R3 microcontroller (U11)	Pass Shot 10 darts and measured their	06/18

	10 m away from the tank.		IRLZ44N MOSFET (Q1) SG90 Servo motor (M1)	distances from the tank. Averaged the distance to be around 12-13m which is over the requirement of 10m.	
HRS-3.1.5	The tank shall have a capacity of at least 10 Nerf darts.			Pass Tank capacity depends on the Nerf magazine used, which can hold up to 10 Nerf darts.	06/18
HRS-3.1.6	The tank shall be able to fire at a maximum fire rate of at least 1 dart per second		Size-130 DC motors (M5 & M7) Arduino Uno R3 microcontroller (U11) IRLZ44N MOSFET (Q1) SG90 Servo motor (M1)	Fail Fire rate is much slower than anticipated Closer to 3 seconds per dart	06/18
HRS-3.1.7	The tank's turret shall be able to rotate continuously, past 360° of rotation	Continuous, infinite rotation	Arduino Uno R3 microcontroller (U11) BYJ-48 Stepper motor (M2) ULN2003 Stepper motor Driver (U8)	Pass Slip ring enables continuous, infinite rotation any direction	06/18
HRS-3.1.8	The tank's turret shall be able to elevate at least 30° upwards from the base position	Base elevation could be below the horizontal (E.g. range of motion from -15° to 15°)	Arduino Uno R3 microcontroller (U11) MG996R Servo motor (M3)	Pass	06/18
HRS-3.1.9	The tank shall include a toggleable red laser beam that indicates where the turret is pointing		Arduino Uno R3 microcontroller (U11) KY-008 Laser Transmitting Module (LD1)	Pass	06/18
HRS-3.1.10	The tank shall be able to independently identify human targets and enter a semi-autonomous mode that tracks targets with its turret	Barrel can move and rotate to aim at identified target independent of user control		Fail Vision processing and Object detection were not incorporated into the design	N/A

Testing of Hardware Requirements

Requirement Specification Reference	Requirement	Comments	Hardware Components Involved in Testing	Status	Date Tested
HRS-3.2.1	The tank's outer body shall be made with 3D-printed parts with PLA	This includes the chassis, barrel, and rotating turret.		Pass	06/18
HRS-3.2.2	The tank's internal mechanisms, including but not limited to flywheels, custom gears, and shafts, shall be made with 3D-printed parts with PETG	Parts that are moving and need to withstand pressure and heat need to be printed with strong, PETG material		Pass with Modification All parts were printed with PLA, which was strong enough Flywheels and flywheel cage were taken from Nerf blaster	06/18
HRS-3.2.3	The tank shall be driven on two rubber tank treads, attached to the body, that are tensioned by free-spinning wheels and powered by gears	Wheels will form the shape and guide the treads, while the gear will drive and physically shift each tread.		Pass with Modification Tank treads are 3D printed with PLA	06/18
HRS-3.2.4	The tank shall have a chassis base that supports the turret and treads	Houses electronic components as well		Pass	06/18
HRS-3.2.5	The tank shall have a turret on top of the chassis for firing	Includes barrel, housing for electronic components		Pass	06/18
HRS-3.2.6	The tank shall be able to accommodate a Nerf magazine, which will be inserted on the side of the tank and can be easily removed or inserted.	3D-printed custom magazine holder Magazine is spring loaded, containing Nerf darts		Pass with Modification Any Nerf magazine is easily inserted on the top of the turret, not the side.	06/18
HRS-3.2.7	The tank shall be powered by a 7.4 V 2S Li-ion Battery		7.4 V 2S Li-ion Battery (BT2)	Pass	06/18
HRS-3.2.8	The tank shall use and contain an Arduino UNO R4		Arduino Uno R3 microcontroller (U11)	Pass with Modification The tank uses an	06/18

	microcontroller. The Arduino will control and power certain input/output devices on the tank.			Arduino Uno 3 microcontroller. The Arduino Uno R4 is not used because they were short-circuited and damaged during previous testing.	
HRS-3.2.9	The tank shall use and contain a Raspberry Pi 4 for vision processing			Fail Design did not incorporate the Raspberry Pi 4 and vision processing because of time constraints.	N/A
HRS-3.2.10	The tank's Raspberry Pi 4 and Arduino Uno R4 microcontrollers shall be connected to a 5 V Buck Converter, which will be connected to the 7.4 V 2S Li-ion Battery.		7.4 V 2S Li-ion Battery (BT2) Arduino Uno R3 microcontroller (U11) XL4016 5 V Buck Converter (U16)	Pass with Modification Arduino Uno R3 microcontroller is connected to a 5V buck converter. No Raspberry Pi 2.	06/18
HRS-3.2.11	The tank shall contain a 28BYJ-48 stepper motor and a ULN2003 driver board for rotating the turret		Arduino Uno R3 microcontroller (U11) BYJ-48 Stepper motor (M2) ULN2003 Stepper motor Driver (U8)	Pass	06/18
HRS-3.2.12	The tank shall contain an MG996R servo motor for changing the turret elevation		Arduino Uno R3 microcontroller (U11) MG996R servo motor (M3)	Pass	06/18
HRS-3.2.13	The tank shall contain a SG90 servo motor for pushing darts to be shot out		Arduino Uno R3 microcontroller (U11) SG90 servo motor (M1)	Pass The SG90 servo motor turns a gear in a rack-and-pinion mechanism to push the darts	06/18
HRS-3.2.14	The tank shall use two DRV8871DDA DC motor drivers to drive the two 6V-rated JGB37-550 37mm motors for moving	Motors have a torque of at least 0.35 Nm and an RPM of around 100-300.	Arduino Uno R3 microcontroller (U11) DRV8871DDA DC motor driver (U14 & U17) JGB37-550 37mm motors (M4 & M6)	Pass	06/18

	the tank tracks				
HRS-3.2.15	The tank shall use one MOSFET to run the flywheel 130-size DC motors	Motors shall be able to reach speeds of at least 25000 rpm within 2-3 seconds, operating continuously for ≥ 60 seconds without overheating	Arduino Uno R3 microcontroller (U11) 130-size DC motors (M5 & M7) IRLZ44N MOSFET (Q1)	Pass with Modification RPM could not be tested reliably, but motors do reach substantial speed within 2-3 seconds and can operate continuously for over 60 seconds without overheating.	06/18
HRS-3.2.16	The tank shall have an nRF24L01+ PA/LNA Radio Module 2.4 GHz for receiving radio signals		Arduino Uno R3 microcontroller (U11) nRF24L01+ PA/LNA Radio Module (U15)	Pass	06/18
HRS-3.2.17	The tank shall have a RunCam Phoenix 2 FPV Camera for analog video to the user		RunCam Phoenix 2 (U12) Eachine TX805 5.8 GHz VTX (U9) Sologood FPV monitor (U1)	Pass	06/18
HRS-3.2.18	The tank shall have an Eachine TX805 5.8 GHz VTX to transmit analog video to the monitor		RunCam Phoenix 2 (U12) Eachine TX805 5.8 GHz VTX (U9) Sologood FPV monitor (U1)	Pass	06/18
HRS-3.2.19	The tank shall have a mounted digital USB AverMedia PW313 Webcam to the turret that is connected to the Raspberry Pi 4 for vision processing			Fail Vision processing and object detection with the digital camera and the Raspberry Pi 4 were not incorporated into the design	N/A
HRS-3.2.20	The controller shall have an Arduino Uno R4 microcontroller		Arduino Uno R3 microcontroller (U2)	Pass with modification: The controller uses an Arduino Uno R3.	06/18
HRS-3.2.21	The controller shall contain a USB Host Shield connecting	USB Host Shield powers and transfers USB data	Arduino Uno R3 microcontroller (U2) USB Host Shield (U3)	Pass	06/18

	the microcontroller to the Xbox controller	between Xbox controller and microcontroller	Xbox One Controller (U5)		
HRS-3.2.22	The controller's Arduino shall be powered by a rechargeable power bank via USB		Arduino Uno R3 microcontroller (U2) 9 V Power Supply (BT1)	Fail The Arduino Uno R3's power supply is 9 V, 6 AA batteries through a barrel jack.	06/18
HRS-3.2.23	The controller shall have a wireless Xbox controller for user control		Xbox One Controller (U5)	Pass Xbox controller is wired to the Arduino Uno R3, which sends radio packets wirelessly to the tank's microcontroller	06/18
HRS-3.2.24	The controller shall have an nRF24L01+ PA/LNA Radio Module 2.4 GHz for transmitting radio signals		Arduino Uno R3 microcontroller (U2) nRF24L01+ PA/LNA Radio Module (U6)	Pass	06/18
HRS-3.2.25	The controller shall contain a drone SoloGood 4.3" FPV monitor for receiving and streaming live video		RunCam Phoenix 2 (U12) Eachine TX805 5.8 GHz VTX (U9) Sologood FPV monitor (U1)	Pass The monitor has an excellent, clear feed that has minimal latency.	06/18
HRS-3.2.26	The controller shall contain a 3D-printed PLA casing to hold the controller's Arduino and power bank, and shall attach the monitor from HRS-3.2.25 to the controller			Pass with Modification The controller has a 3D printed container that holds the Sologood FPV monitor and the Xbox controller together. The Arduino, radio module, and 6 AA batteries as the 9 V power supply are glued onto the back of the monitor.	06/18

Testing of User Interface Requirements

Requirement Specification Reference	Requirement	Comments	Hardware Components Involved	Status	Date Tested
--	--------------------	-----------------	-------------------------------------	---------------	--------------------

			in Testing		
HRS-3.3.1	The tank shall be controlled by a user-operated remote Xbox controller that sends 2.4 GHz radio signals to control the tank.	Using nRF24L01+ PA/LNA radio modules, with Arduino UNO R4s, USB Host Shield, and standard Xbox wireless controllers The user can control the tank remotely	Xbox One controller (U5) Arduino Uno R3 microcontroller (U2 & U11) nRF24L01+ PA/LNA radio module (U6 & U15) USB Host Shield (U3)	Pass	06/18
HRS-3.3.2	The tank shall stream live, analog video through wireless 5.8 GHz radio signals to an FPV monitor	Monitor is attached to the controller The user can watch the live video stream of the tank while they are controlling it	Eachine TX805 5.8 GHz VTX (U9) Sologood FPV monitor (U1) RunCam Phoenix 2 (U12)	Pass	06/18
HRS-3.3.3	The tank shall be able to move its treads via joystick, radio frequency remote control	Tank drive, using an arcade drive control system where one joystick controls both tank treads Left joystick on Xbox controller	Xbox One controller (U5) Arduino Uno R3 microcontroller (U2 & U11) nRF24L01+ PA/LNA radio module (U6 & U15) USB Host Shield (U3) JGB37-550 37mm motors (M4 & M6) DRV8871DDA DC Motor Driver (U14 & U17)	Pass	06/18
HRS-3.3.4	The tank shall be able to spin the flywheels when the user holds a trigger down on a radio frequency remote controller.	Flywheels will spin at different speeds based on how much the trigger is held down Left trigger on Xbox controller	Xbox One controller (U5) Arduino Uno R3 microcontroller (U2 & U11) nRF24L01+ PA/LNA radio module (U6 & U15) USB Host Shield (U3) Size-130 DC motors (M5 & M7) IRLZ44N MOSFET (Q1)	Pass with Modification As outlined in HRS-3.4.2, flywheel motors only spin at max speed.	06/18

HRS-3.3.5	The tank shall be able to shoot a dart when the user presses a button on a radio frequency remote controller. If the user holds the button, the tank shall be able to continuously fire darts. This feature will only work if flywheels are activated as outlined in HRS-3.3.4	Right trigger on Xbox controller	Xbox One controller (U5) Arduino Uno R3 microcontroller (U2 & U11) nRF24L01+ PA/LNA radio module (U6 & U15) USB Host Shield (U3) Size-130 DC motors (M5 & M7) IRLZ44N MOSFET (Q1) SG90 Servo motor (M1)	Pass	06/18
HRS-3.3.6	The turret on the tank shall be able to rotate when the user moves the joystick of a radio frequency remote control left and right. The barrel on the tank shall be able to move up and down when the user moves the same joystick up and down.	Left/right yaw movement and up/down pitch movement on same joystick Right joystick on Xbox controller	Xbox One controller (U5) Arduino Uno R3 microcontroller (U2 & U11) nRF24L01+ PA/LNA radio module (U6 & U15) USB Host Shield (U3) BYJ-48 Stepper motor ULN2003 Stepper motor driver MG996R Servo motor	Pass with Modification Left/right yaw movement on the right joystick Up/down pitch movement on up/down D-pads	06/18
HRS-3.3.7	The tank shall be able to toggle a laser on or off with a radio frequency remote controller. The laser should indicate where the dart will hit.	Face button X on the Xbox controller	Xbox One controller (U5) Arduino Uno R3 microcontroller (U2 & U11) nRF24L01+ PA/LNA radio module (U6 & U15) USB Host Shield (U3) KY-008 Laser Transmitting Module (LD1)	Pass Not perfect accuracy regarding the Nerf dart, but functionality remains.	06/18
HRS-3.3.8	The tank shall be able to enter and exit an autonomous mode once a user toggles a button on a radio frequency remote controller. The autonomous mode allows the tank to move and	Face button A on the Xbox controller		Fail Autonomous control and vision processing were not incorporated in the final design.	

	aim at targets independent of user control, but it will not be able to fire.				
HRS-3.3.9	The tank shall automatically power off and cease movement if it loses communication with the remote controller	If signals become distorted or the tank stops receiving signals, such as if something breaks or the tank goes out of range, the tank stops all movements	Arduino Uno R3 (U11) nRF24L01+ PA/LNA radio module (U15)	Pass Tested Safe Stop function that consistently pauses all Arduino signals until radio communication is reestablished.	06/18
HRS-3.3.10	The tank shall have a physical on/off switch that allows users to power/start the tank and power off the tank.	Switch to connect or disconnect battery	Toggle Switch (SW1)	Pass	06/18
HRS-3.3.11	The tank shall be able to fire Nerf darts at users and non-user targets and hit them	Physical user-system interaction	Arduino Uno R3 microcontroller (U11) Size-130 DC motors (M5 & M7) IRLZ44N MOSFET (Q1) SG90 Servo motor (M1)	Pass Able to shoot Nerf darts and hit targets.	06/18

Testing of Hardware Interface Requirements

Requirement Specification Reference	Requirement	Comments	Hardware Components Involved in Testing	Status	Date Tested
HRS-3.4.1	The tank's Arduino microcontroller shall drive both DRV8871DDA DC motor drivers with a PWM signal, which in turn will drive 2 6V-rated JGB37-550 37mm motors connected to gears, which will drive the tank treads		Arduino Uno R3 microcontroller (U11) JGB37-550 37mm motors (M4 & M6) DRV8871DDA DC Motor Driver (U14 & U17)	Pass Tested driving of tank treads in both directions on the ground	06/18

HRS-3.4.2	The tank's Arduino microcontroller shall drive the MOSFET with a PWM signal, which will drive two 130-Size High-RPM DC Motors, which will be connected to the flywheels.		Arduino Uno R3 microcontroller (U11) Size-130 DC motors (M5 & M7) IRLZ44N MOSFET (Q1)	Pass with Modification The tank's Arduino microcontroller drives the MOSFET with a digital signal, which drives two 130-Size DC motors. This is because on the Arduino Uno R3, all the PWM pins are used by the drivetrain motors and SPI communication, so this connection was moved to digital pin 7. Minimal functionality lost.	06/18
HRS-3.4.3	The tank's Arduino microcontroller shall drive the laser module with a digital output signal		Arduino Uno R3 microcontroller (U11) KY-008 Laser Transmitting Module (LD1)	Pass	06/18
HRS-3.4.4	The Arduino microcontroller shall control a stepper driver that will drive a stepper motor that will be used to rotate the turret	Logic inputs from the microcontroller into the driver determining speed, direction, enabling, step resolution The driver controls the motor by phase power switching that connects 5V to ground on specified phases, individual ends of the motor coils.	Arduino Uno R3 microcontroller (U11) BYJ-48 Stepper motor (M2) ULN2003 Stepper motor driver (U8)	Pass The stepper motor is much slower than expected, with an approximate turret rotation speed of 1 rotation every 8 seconds.	06/18
HRS-3.4.5	The tank's Arduino microcontroller shall drive a servo motor with a PWM signal that will change the elevation of the barrel.		Arduino Uno R3 microcontroller (U11) MG996R Servo motor (M3)	Pass	06/18
HRS-3.4.6	The tank's Arduino microcontroller shall drive a servo motor with a PWM signal that will push the dart into the flywheels		Arduino Uno R3 microcontroller (U11) SG90 Servo motor (M1)	Pass	06/18

HRS-3.4.5	The tank's Arduino UNO R4 communicates with the Raspberry Pi 4 by Serial UART over USB			Fail The Raspberry Pi 4 was not incorporated into the final design.	N/A
HRS-3.4.7	The tank shall have a digital webcam connected to a Raspberry Pi 4 via USB for vision processing.			Fail The Raspberry Pi 4 was not incorporated into the final design.	N/A
HRS-3.4.8	The tank's Arduino microcontroller shall receive and interpret radio signals with an nRF24L01+PA+LNA radio module by SPI		Arduino Uno R3 microcontroller (U11) nRF24L01+ PA/LNA radio module (U15)	Pass Packets were programmed to contain a specific sequence of information and address. Had to use low power mode to avoid loss of signal integrity	06/18
HRS-3.4.9	The controller's Arduino microcontroller shall transmit and package radio signals with an nRF24L01+PA/LNA radio module by SPI		Arduino Uno R3 microcontroller (U2) nRF24L01+ PA/LNA radio module (U6) USB Host Shield (U3)	Pass Packets were programmed to contain a specific sequence of information and address Had to use low power mode to avoid loss of integrity	06/18
HRS-3.4.10	The tank's radio module shall receive transmitted radio signals from the controller's radio module through wireless 2.4 GHz radio signals	Radio waves will be modulated and customized data packets Antenna-to-antenna communication	Arduino Uno R3 microcontroller (U2 & U11) nRF24L01+ PA/LNA radio module (U6 & U15)	Pass Tested with example sketches of both transceivers communicating.	06/18
HRS-3.4.11	The controller's Arduino microcontroller shall interface with a USB Host Shield by SPI	Note: both the radio module and host shield will need to share some SPI pins on the controller's microcontroller, but differ in address pins	Arduino Uno R3 microcontroller (U2) USB Host Shield (U3)	Pass	06/18

HRS-3.4.12	The controller's USB Host Shield will connect the Xbox Controller to the microcontroller by USB		XBox One controller (U5) Arduino Uno R3 microcontroller (U2) USB Host Shield (U3)	Pass	06/18
HRS3.4.13	The tank shall have an FPV camera connected to a 5.8 GHz VTX transmitter	Camera sends analog composite video to the VTX	Eachine TX805 5.8 GHz VTX (U9) RunCam Phoenix 2 (U12)	Pass	06/18
HRS3.4.14	The tank's video transmitter will send 5.8 GHz video signal to an FPV monitor on the controller	Analog video transmitted as radio waves Monitor and VTX need to be set to be on the same channel (frequency) VTX can switch power of signal (25 mw, 200 mW, 600 mW, 800 mW), changing range, power usage, and signal integrity	Eachine TX805 5.8 GHz VTX (U9) Sologood FPV Monitor (U1) RunCam Phoenix 2 (U12)	Pass Had to use 25 mW power mode because the other modes would be too hot	06/18

Appendix B: Reflection of Learning

Appendix C: Code

Tank Code

```
// =====
// Sentinel Tank
// Bryan Zhang, Bruce Zhang
// June 19, 2026
//
// A program designed to receive radio signals containing data from the Sentinel
// controller, translating the data into actuator and motor outputs, controlling
// the tank according to user control.
//
// Hardware:
//   - Arduino UNO R3 (U11)
//   - KY-008 Laser Transmitting Module (LD1)
//   - SG90 Servo Motor (M1)
//   - BYJ-48 Stepper Motor (M2)
//   - MG996R Servo Motor (M3)
//   - JGB37-550 37mm DC Motors (M4 & M6)
//   - Nerf Size-130 DC Motors (M5 & M7)
// =====
#include <SPI.h>
```

```

#include <RF24.h>
#include <Servo.h>
#include <AccelStepper.h>

// -- Pin definitions -----
#define NRF_CSN_PIN 4
#define NRF_CE_PIN 8

#define MOTOR_A_IN1 5
#define MOTOR_A_IN2 3
#define MOTOR_B_IN1 9
#define MOTOR_B_IN2 6
#define FLYWHEEL_PIN 7
#define LASER_PIN 2

#define SHOVER_PIN A4
#define BARREL_PIN A5

#define STEP_PIN1 A0
#define STEP_PIN2 A1
#define STEP_PIN3 A2
#define STEP_PIN4 A3

// -- Buttons -----
#define BTN_X (1 << 1)
#define BTN_DPAD_UP (1 << 6)
#define BTN_DPAD_DOWN (1 << 7)

// -- Payload -----
struct Payload {
    int16_t leftX;
    int16_t leftY;
    int16_t rightX;
    int16_t rightY;
    uint8_t leftTrig;
    uint8_t rightTrig;
    uint8_t buttons;
};

// -- Forward declaration -----
void handlePacket(const Payload &pkt, unsigned long now);

// -- Radio -----
RF24 radio(NRF_CE_PIN, NRF_CSN_PIN);
const byte ADDRESS[6] = "SNTL1";

// -- Servos -----
Servo shoverServo;
Servo barrelServo;

// -- Stepper -----
#define STEPS_PER_REV 2048
AccelStepper turretStepper(AccelStepper::HALF4WIRE, STEP_PIN1, STEP_PIN3, STEP_PIN2, STEP_PIN4);

// -- State -----
bool laserOn = false;
bool flywheelsOn = false;
bool shootActive = false;

int barrelAngle = 15;

```

```

const int BARREL_MIN = 0;
const int BARREL_MAX = 30;

long turretTarget = 0;

// -- timing -----
unsigned long lastPacketTime = 0;
const unsigned long WATCHDOG_MS = 1000;

unsigned long lastShoveTime = 0;
const unsigned long SHOVE_INTERVAL = 500;
bool shoverState = false;

// -- startup safety system -----
bool systemReady = false;
uint8_t packetCount = 0;
uint8_t lastButtons = 0;

void setMotor(uint8_t in1, uint8_t in2, int speed) {
    speed = constrain(speed, -255, 255);

    if (speed >= 0) {
        // Forward: IN1 handles direction (flat HIGH), IN2 handles speed via inverted PWM
        // This allows Pin 9 (IN1) to act as a pure digital pin
        digitalWrite(in1, HIGH);
        analogWrite(in2, 255 - speed);
    }
    else {
        // Backward: IN1 goes LOW, IN2 handles raw speed PWM
        digitalWrite(in1, LOW);
        analogWrite(in2, -speed);
    }
}

// -- Arcade drive -----
void arcadeDrive(int16_t joyY, int16_t joyX, int &leftOut, int &rightOut) {
    int throttle = map((long)joyY, -32767L, 32767L, -255L, 255L);
    int turn     = map((long)joyX, -32767L, 32767L, -255L, 255L);

    leftOut  = constrain(throttle + turn, -255, 255);
    rightOut = constrain(throttle - turn, -255, 255);
}

// -- safe stop -----
void safeStop() {
    setMotor(MOTOR_A_IN1, MOTOR_A_IN2, 0);
    setMotor(MOTOR_B_IN1, MOTOR_B_IN2, 0);
    analogWrite(FLYWHEEL_PIN, 0);

    shoverServo.write(0);
    digitalWrite(LASER_PIN, LOW);

    laserOn = false;
    flywheelsOn = false;
    shootActive = false;
}

// -- setup -----
void setup() {

```

```

pinMode(MOTOR_A_IN1, OUTPUT);
pinMode(MOTOR_A_IN2, OUTPUT);
pinMode(MOTOR_B_IN1, OUTPUT);
pinMode(MOTOR_B_IN2, OUTPUT);

pinMode(FLYWHEEL_PIN, OUTPUT);
pinMode(LASER_PIN, OUTPUT);

// immediate safe servo position —
delay(300);
shoverServo.write(0);
barrelServo.write(barrelAngle);
shoverServo.attach(SHOVER_PIN);
barrelServo.attach(BARREL_PIN);

safeStop();

turretStepper.setMaxSpeed(1000);
turretStepper.setAcceleration(400);

radio.begin();
radio.setPALevel(RF24_PA_LOW);
radio.setDataRate(RF24_250KBPS);
radio.setChannel(76);
radio.openReadingPipe(1, ADDRESS);
radio.startListening();

delay(1000); // full power stabilization delay

systemReady = true;
lastPacketTime = millis();
}

// — Loop —————
void loop() {

    if (!systemReady) return;

    unsigned long now = millis();

    if (now - lastPacketTime > WATCHDOG_MS) {
        safeStop();
    }

    Payload pkt;
    if (radio.available()) {
        radio.read(&pkt, sizeof(pkt));
        lastPacketTime = now;
        handlePacket(pkt, now);
    }

    // — shover —————
    if (shootActive && flywheelsOn) {
        if (now - lastShoveTime > SHOVE_INTERVAL) {
            lastShoveTime = now;
            shoverState = !shoverState;
            shoverServo.write(shoverState ? 180 : 0);
        }
    } else {
        shoverServo.write(0);
    }
}

```

```

    shoverState = false;
}

// -- turret -----
turretStepper.run();
}

// -- packet handler -----
void handlePacket(const Payload &pkt, unsigned long now) {

    // -- ignore first packets -----
    if (packetCount < 10) {
        packetCount++;
        lastButtons = pkt.buttons;
        return;
    }

    uint8_t rising = pkt.buttons & ~lastButtons;

    if (rising & BTN_X) {
        laserOn = !laserOn;
        digitalWrite(LASER_PIN, laserOn);
    }

    // -- barrel control -----
    if (pkt.buttons & BTN_DPAD_UP)
        barrelAngle = constrain(barrelAngle + 3, BARREL_MIN, BARREL_MAX);

    if (pkt.buttons & BTN_DPAD_DOWN)
        barrelAngle = constrain(barrelAngle - 3, BARREL_MIN, BARREL_MAX);

    barrelServo.write(barrelAngle);

    // -- drive -----
    int leftSpeed, rightSpeed;
    arcadeDrive(pkt.leftY, pkt.leftX, leftSpeed, rightSpeed);

    setMotor(MOTOR_A_IN1, MOTOR_A_IN2, leftSpeed);
    setMotor(MOTOR_B_IN1, MOTOR_B_IN2, -rightSpeed);

    // -- turret input -----
    if (abs(pkt.rightX) > 7500) {
        turretTarget += map(pkt.rightX, -32767, 32767, -20, 20);
    }
    turretStepper.moveTo(turretTarget);

    // -- flywheels -----
    flywheelsOn = (pkt.leftTrig > 10);

    if (flywheelsOn) {
        digitalWrite(FLYWHEEL_PIN, HIGH);
    } else {
        digitalWrite(FLYWHEEL_PIN, LOW);
    }

    shootActive = (flywheelsOn && pkt.rightTrig > 10);

    lastButtons = pkt.buttons;
}

```

Controller Code

```
// =====
// Sentinel Controller
// Bryan Zhang, Bruce Zhang
// June 19, 2026
//
// A program designed to transform user inputs on an Xbox Controller into radio
// information to be sent over a 2.4 GHz channel communication link to the Sentinel,
// allowing user control of the tank.
//
// Hardware:
//   - Arduino UNO R3 (U2)
//   - USB Host Shield (U3) → D10 CS, D11 MOSI, D12 MISO, D13 SCK, D2 INT
//   - Xbox One Controller (U5) → USB Host Shield
//   - nRF24L01+PA+LNA (U6) → D9 CSN, D8 CE, D11 MOSI, D12 MISO, D13 SCK
//
// Control Scheme:
//   Left Joystick Y → Drive forward / backward
//   Left Joystick X → Pivot / turn (tank drive)
//   Right Joystick X → Rotate turret
//   Left Trigger → Rev flywheels (analog speed)
//   Right Trigger → Shoot dart (only fires if flywheels spinning)
//   X → Toggle laser
//   D-Pad U/D → Barrel elevation setpoints (3° increments)
// =====

#include <XBOXONE.h>
#include <SPI.h>
#include <RF24.h>

// -- nRF24L01 pins -----
#define NRF_CE_PIN 8
#define NRF_CSN_PIN 4

// -- Radio -----
RF24 radio(NRF_CE_PIN, NRF_CSN_PIN);
const byte ADDRESS[6] = "SNTL1";

// -- USB / Xbox -----
USB Usb;
XBOXONE Xbox(&Usb);

// -- Payload struct (max 32 bytes) -----
// Total: 2+2+2+2+1+1+1 = 11 bytes
struct Payload {
    int16_t leftX;      // -32768 to 32767
    int16_t leftY;
    int16_t rightX;
    int16_t rightY;
    uint8_t leftTrig;   // 0-255
    uint8_t rightTrig;  // 0-255
    uint8_t buttons;
};

// -- Button bitmask positions -----
#define BTN_X (1 << 1)
#define BTN_DPAD_UP (1 << 6)
#define BTN_DPAD_DOWN (1 << 7)

// -- Deadband -----
```

```

#define DEADBAND 7500

// --- Exponential curve helper -----
// Applies a mild square curve while preserving sign.
// Input/output: -32767 to 32767
int16_t expoCurve(int16_t raw) {
    if (abs(raw) < DEADBAND) return 0;
    float n = (float)raw / 32767.0f;
    // blend: 50% linear + 50% cubic
    float curved = 0.5f * n + 0.5f * n * n * n;
    return (int16_t)(curved * 32767.0f);
}

// --- Setup -----
void setup() {
    Serial.begin(115200);

    // force both spi devices silent immediately
    pinMode(10, OUTPUT);
    digitalWrite(10, HIGH); // De-select USB Host Shield
    pinMode(NRF_CSN_PIN, OUTPUT);
    digitalWrite(NRF_CSN_PIN, HIGH); // De-select nRF24L01
    delay(100);

    // initialize usb shield
    Serial.println(F("Initializing USB Host Shield..."));
    if (Usb.Init() == -1) {
        Serial.println(F("USB Host Shield did not start"));
        while (1);
    }
    digitalWrite(10, HIGH); // Ensure it releases the SPI bus immediately after
    Serial.println(F("USB Host Shield started"));

    // initialize nrf24l01
    Serial.println(F("Initializing nRF24L01..."));
    if (!radio.begin()) {
        Serial.println(F("nRF24L01 not found"));
        while (1);
    }
    radio.setPALevel(RF24_PA_LOW);
    radio.setDataRate(RF24_250KBPS);
    radio.setChannel(76);
    radio.openWritingPipe(ADDRESS);
    radio.stopListening();

    digitalWrite(NRF_CSN_PIN, HIGH); // Ensure radio releases the bus
    Serial.println(F("nRF24L01 ready - Sentinel Controller online"));

    Xbox.setRumbleOff();
}

// --- State for non-blocking transmission timer -----
unsigned long lastTxTime = 0;
const unsigned long TX_INTERVAL_MS = 20;

// --- Loop -----
void loop() {
    Usb.Task();

    if (!Xbox.XboxOneConnected) return;

```



```

// -- Read raw joystick values -----
int16_t rawLX = Xbox.getAnalogHat(LeftHatX);
int16_t rawLY = Xbox.getAnalogHat(LeftHatY);
int16_t rawRX = Xbox.getAnalogHat(RightHatX);
int16_t rawRY = Xbox.getAnalogHat(RightHatY);

// Apply deadband + expo curve
int16_t lx = expoCurve(rawLX);
int16_t ly = expoCurve(rawLY);
int16_t rx = expoCurve(rawRX);
int16_t ry = expoCurve(rawRY);

// -- Triggers -----
uint8_t lt = (uint8_t)map(Xbox.getButtonPress(LT), 0, 1023, 0, 255);
uint8_t rt = (uint8_t)map(Xbox.getButtonPress(RT), 0, 1023, 0, 255);

// -- Button clicks -----
static uint8_t accumulatedButtons = 0;

// Check clicks continuously on every fast loop pass
if (Xbox.getButtonClick(X))      accumulatedButtons |= BTN_X;
if (Xbox.getButtonClick(UP))    accumulatedButtons |= BTN_DPAD_UP;
if (Xbox.getButtonClick(DOWN))  accumulatedButtons |= BTN_DPAD_DOWN;

// -- Non-Blocking Transmission Engine -----
unsigned long now = millis();
if (now - lastTxTime >= TX_INTERVAL_MS) {
    lastTxTime = now;

    // Pack the payload using our accumulated buttons
    Payload pkt;
    pkt.leftX      = lx;
    pkt.leftY      = ly;
    pkt.rightX     = rx;
    pkt.rightY     = ry;
    pkt.leftTrig   = lt;
    pkt.rightTrig  = rt;
    pkt.buttons    = accumulatedButtons;
    // Send data over nRF24
    bool ok = radio.write(&pkt, sizeof(pkt));
    // Clear the accumulator so we can collect fresh clicks for the next packet
    accumulatedButtons = 0;

}
}

```